Data Analytics and Computational Social Science Program

Data Visualization

Prof. José Manuel MAGALLANES

Text Data

Text can also be visualized, but unlike numeric variables, text usually needs extra preparation before plotting. Words appear in different forms, some words are very common but not very informative, and some meaningful ideas are expressed as phrases rather than single words.

In this session, you will learn a simple workflow for visualizing text data. We will begin with word frequencies, then compare different graphical options:

- word clouds
- ranked bar charts
- lollipop charts
- treemaps
- topic models
- co-occurrence networks

The goal is not only to produce attractive plots, but also to understand what each plot is good for.

1. Get the text:

Let me get a data frame with texts from some tweets:

```
# clean memory
rm(list = ls())
link1="https://github.com/DACSS-Visual/basicText/"
link2="raw/main/trumps.csv"

trumpLink=paste0(link1,link2)
allTweets=read.csv(trumpLink ,stringsAsFactors = F)
```

The data has these columns:

```
names(allTweets)
```

```
## [1] "created_at"     "text"           "is_retweet"     "favorite_count"
## [5] "retweet_count"  "Hour"           "Day"            "Date"
```

The Twitter text column has this information:

```
head(allTweets$text,2)
```

```
## [1] ".@DonYoungAK really produces for Alaska. He is an incredible Congressman who lov
es his State and works tirelessly for it. Strong on Public Lands, Energy, and our Second
Amendment. He will never let you down, and has my Complete and Total Endorsement! http
s://t.co/cUR3zuUfs6"
## [2] "...She will be a great Senator, and has my Complete and Total Endorsement! http
s://t.co/7yHPhxqQVS"
```

2. Select the observations of interest

This data frame has several columns that allow subsetting. In this example, I will keep only tweets that are not retweets:

```
SomeTrumpTweets = allTweets[allTweets$is_retweet == FALSE, ]
SomeTrumpTweets = SomeTrumpTweets[, c("created_at", "text")]
# see
head(SomeTrumpTweets, 3)
```

```
##            created_at
## 1 2020-08-13 23:26:50
## 2 2020-08-13 23:23:26
## 3 2020-08-13 23:23:25
##
text
## 1 .@DonYoungAK really produces for Alaska. He is an incredible Congressman who loves
his State and works tirelessly for it. Strong on Public Lands, Energy, and our Second Am
endment. He will never let you down, and has my Complete and Total Endorsement! https://
t.co/cUR3zuUfs6
## 2
...She will be a great Senator, and has my Complete and Total Endorsement! https://t.co/
7yHPhxqQVS
## 3                                                                    .@CynthiaMLummis i
s a friend of mine and a great woman. She is running for Senate in the very Special Stat
e of Wyoming. Cynthia is Strongly for our Military, our Vets, and protection of the Seco
nd Amendment...
```

At this point, we still have full text messages. To visualize text, we first need to break each message into smaller units.

3. Turn text into words

This process is called **tokenization**. Here, each tweet will be split into individual words.

```
library(dplyr)
library(tidytext)

WordsIn_SomeTrumpTweets = SomeTrumpTweets |>
                 unnest_tokens(output=EachWord, # column created
                               input=text,# input column from SomeTrumpTweets
                               token="words") # level of unnesting

# the date is the same for all the words coming from the
# same tweet:
head(WordsIn_SomeTrumpTweets,10) # notice 'EachWord'
```

```
##             created_at    EachWord
## 1  2020-08-13 23:26:50  donyoungak
## 2  2020-08-13 23:26:50      really
## 3  2020-08-13 23:26:50    produces
## 4  2020-08-13 23:26:50         for
## 5  2020-08-13 23:26:50      alaska
## 6  2020-08-13 23:26:50          he
## 7  2020-08-13 23:26:50          is
## 8  2020-08-13 23:26:50          an
## 9  2020-08-13 23:26:50  incredible
## 10 2020-08-13 23:26:50 congressman
```

Notice that the date is repeated for every word that comes from the same tweet.

How many word-tokens do we have now?

```
nrow(WordsIn_SomeTrumpTweets) # count of words
```

```
## [1] 3028
```

This number includes repeated words, which is useful because frequency is often our first summary of text.

4. Remove common words: **stopwords**

Many very frequent words carry little substantive meaning. Words such as the, and, of, or to are useful for grammar, but not always useful for analysis. These are called *stopwords*.

```
# calling the file
data(stop_words) # from library 'tidytext'
# seeing some 'STOP WORDS'
head(stop_words)
```

```
## # A tibble: 6 × 2
##   word      lexicon
##   <chr>     <chr>
## 1 a         SMART
## 2 a's       SMART
## 3 able      SMART
## 4 about     SMART
## 5 above     SMART
## 6 according SMART
```

Then, we remove the stop words from the *EachWord* column:

```
library(dplyr)

 # The column 'word' from 'stop_words' will be compared # to the column 'EachWord' in 'W
ordsIn_SomeTrumpTweets'
WordsIn_SomeTrumpTweets = WordsIn_SomeTrumpTweets |>anti_join(stop_words,
                                                 by = c("EachWord" = "word"))

# You have these many rows now:

nrow(WordsIn_SomeTrumpTweets) # less than 50%
```

```
## [1] 1496
```

After removing stopwords, the number of rows decreases because many very common words are filtered out.

This does not mean the remaining words are **per se** important. It only means we are removing words that are usually too general to help interpretation.

5. Compute **frequency** of each word:

Here, you are simply producing a frequency table:

```
forCloud = WordsIn_SomeTrumpTweets |>
  count(EachWord, name = "Counts", sort = TRUE)

head(forCloud, 10)
```

```
##          EachWord Counts
## 1           https     54
## 2            t.co     54
## 3             bus     20
## 4          people     18
## 5           usdot     17
## 6  infrastructure     14
## 7         service     14
## 8         support     13
## 9        complete     12
## 10           news     12
```

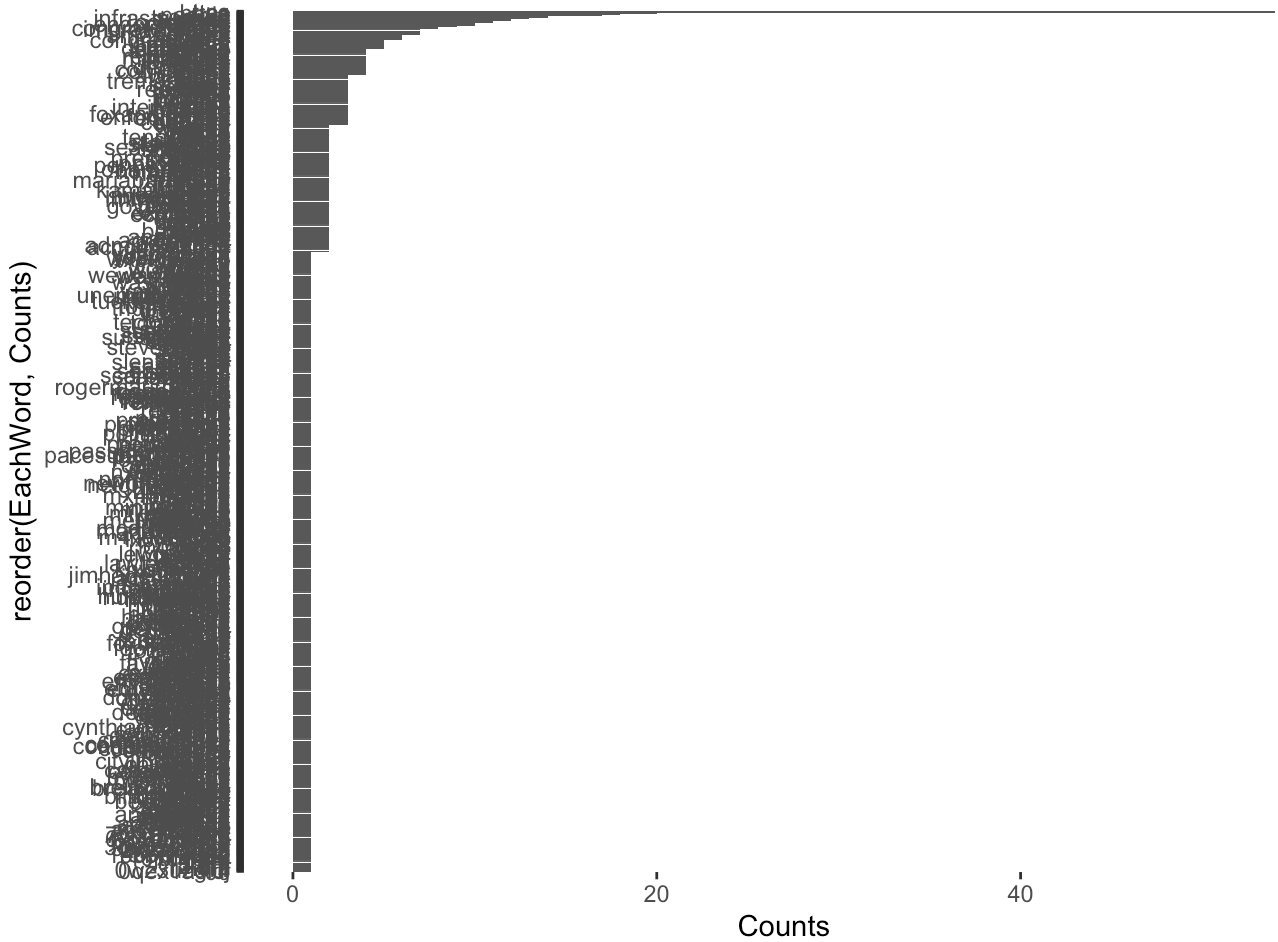
The most frequent words appear at the top.

Since you have a frequency table, you could prepare a barplot:

```
library(ggplot2)
```

```
## Warning: package 'ggplot2' was built under R version 4.5.2
```

```
ggplot(data=forCloud) + geom_bar(aes(x=reorder(EachWord,Counts), y=Counts), stat = 'iden
tity') + coord_flip()
```



However, text from tweets often contains noisy elements such as shortened links or platform-specific tokens. So some additional cleaning may still be necessary.

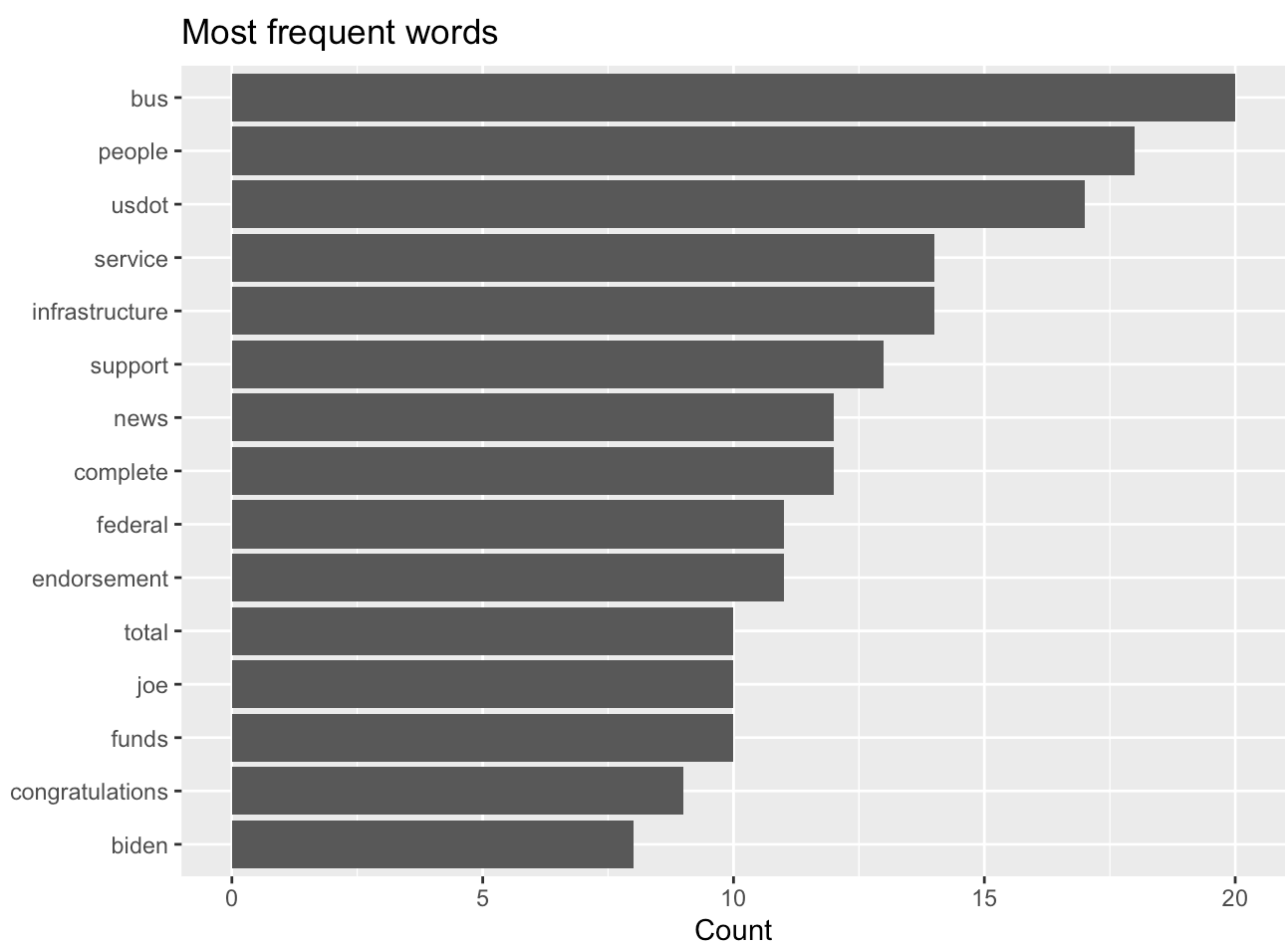
```
badWords = c("https", "t.co")

forCloud = forCloud |>
  filter(!EachWord %in% badWords) |>  # dropping by text:
  filter(Counts > 4) # dropping by count:

head(forCloud, 10)
```

```
##          EachWord Counts
## 1             bus     20
## 2          people     18
## 3           usdot     17
## 4  infrastructure     14
## 5         service     14
## 6         support     13
## 7        complete     12
## 8            news     12
## 9     endorsement     11
## 10        federal     11
```

6. Ranked bar chart

Let's re create the bar chart, just for the top 15 rows.

```
library(ggplot2)

ggplot(forCloud |> slice_head(n = 15),
       aes(x = reorder(EachWord, Counts), y = Counts)) +
  geom_col() +
  coord_flip() +
  labs(x = "", y = "Count",
       title = "Most frequent words")
```



This plot is very useful because it allows precise comparison. You can clearly see:

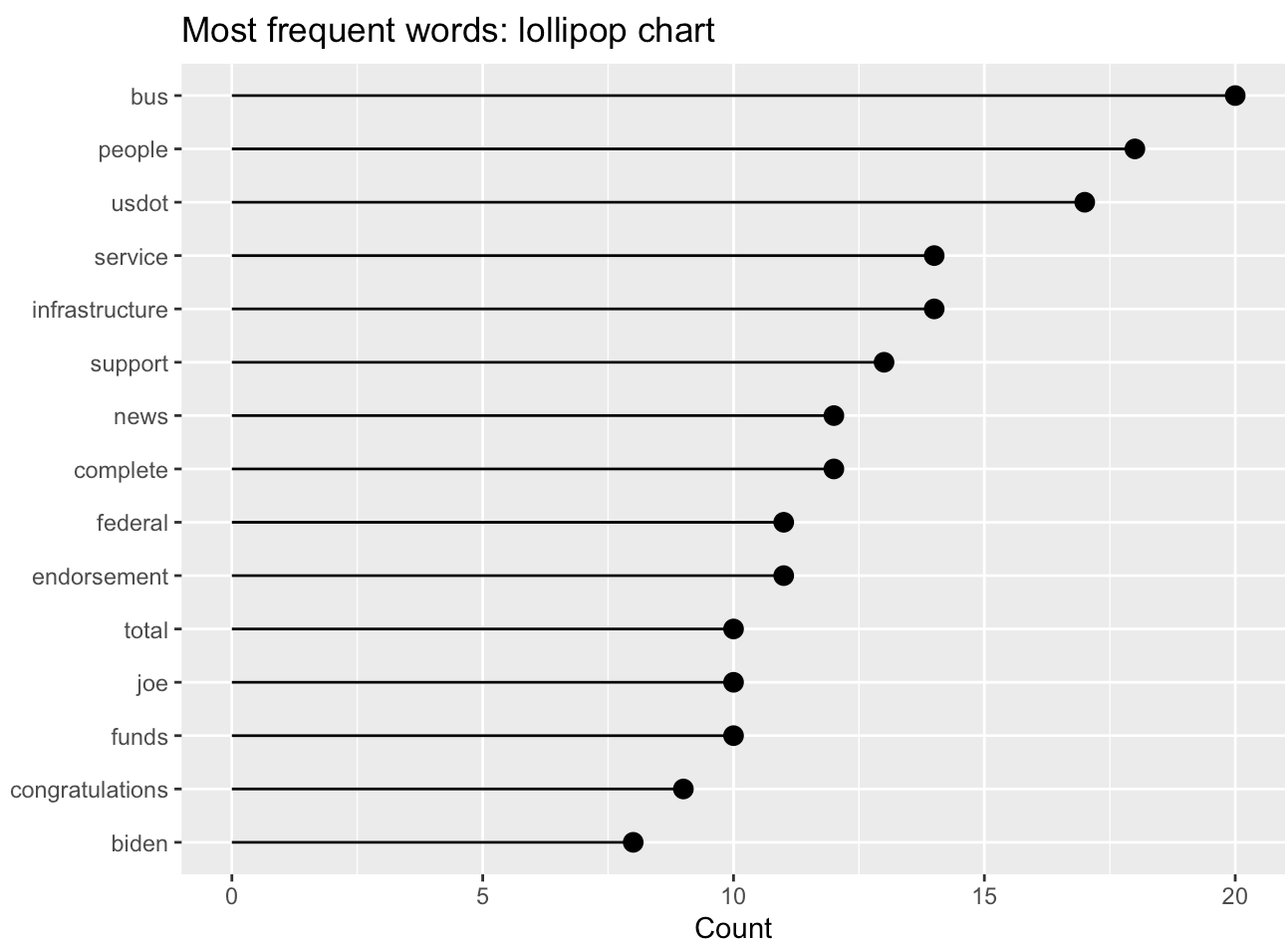
- which words are most frequent
- their ranking
- the size of the differences between them

> Notice that for comparing frequencies, bar charts are usually better than word clouds.

7. Lollipop chart

A lollipop chart shows the same information as a bar chart, but with a lighter appearance.

```
ggplot(forCloud |> slice_head(n = 15),
       aes(x = reorder(EachWord, Counts), y = Counts)) +
  geom_segment(aes(xend = EachWord, y = 0, yend = Counts)) +
  geom_point(size = 3) +
  coord_flip() +
  labs(x = "", y = "Count",
       title = "Most frequent words: lollipop chart")
```

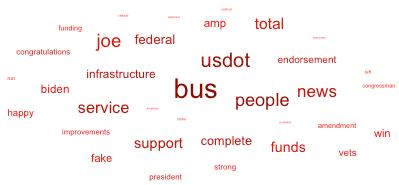


This is useful when you want a cleaner look but still need a ranked comparison.

8. Word cloud

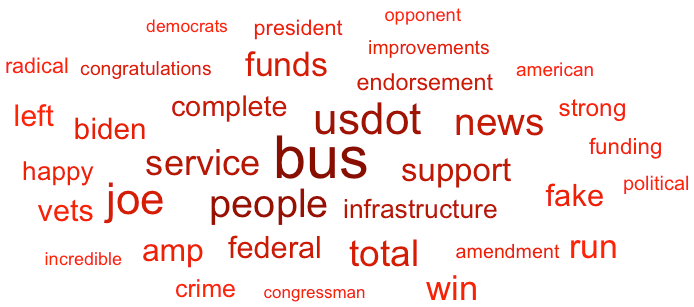
Now let us build the word cloud using **ggwordcloud**.

```
library(ggwordcloud)

ggplot(forCloud,
       aes(label = EachWord,
           size = Counts,
           color = Counts)) +
  geom_text_wordcloud_area() +
  theme_minimal() +
  scale_color_gradient(low = "red", high = "darkred")
```



You can improve this using **scale_size_area()**:

```
ggplot(forCloud,
       aes(label = EachWord,
           size = Counts,
           color = Counts)) +
  geom_text_wordcloud_area(eccentricity = 0.65) +
  theme_minimal() +
  scale_size_area(max_size = 13) +
  scale_color_gradient(low = "red", high = "darkred")
```



Word clouds are often visually attractive, but they are not ideal for accurate comparison. They are better for quick engagement and general impression than for careful measurement.

9. Treemap

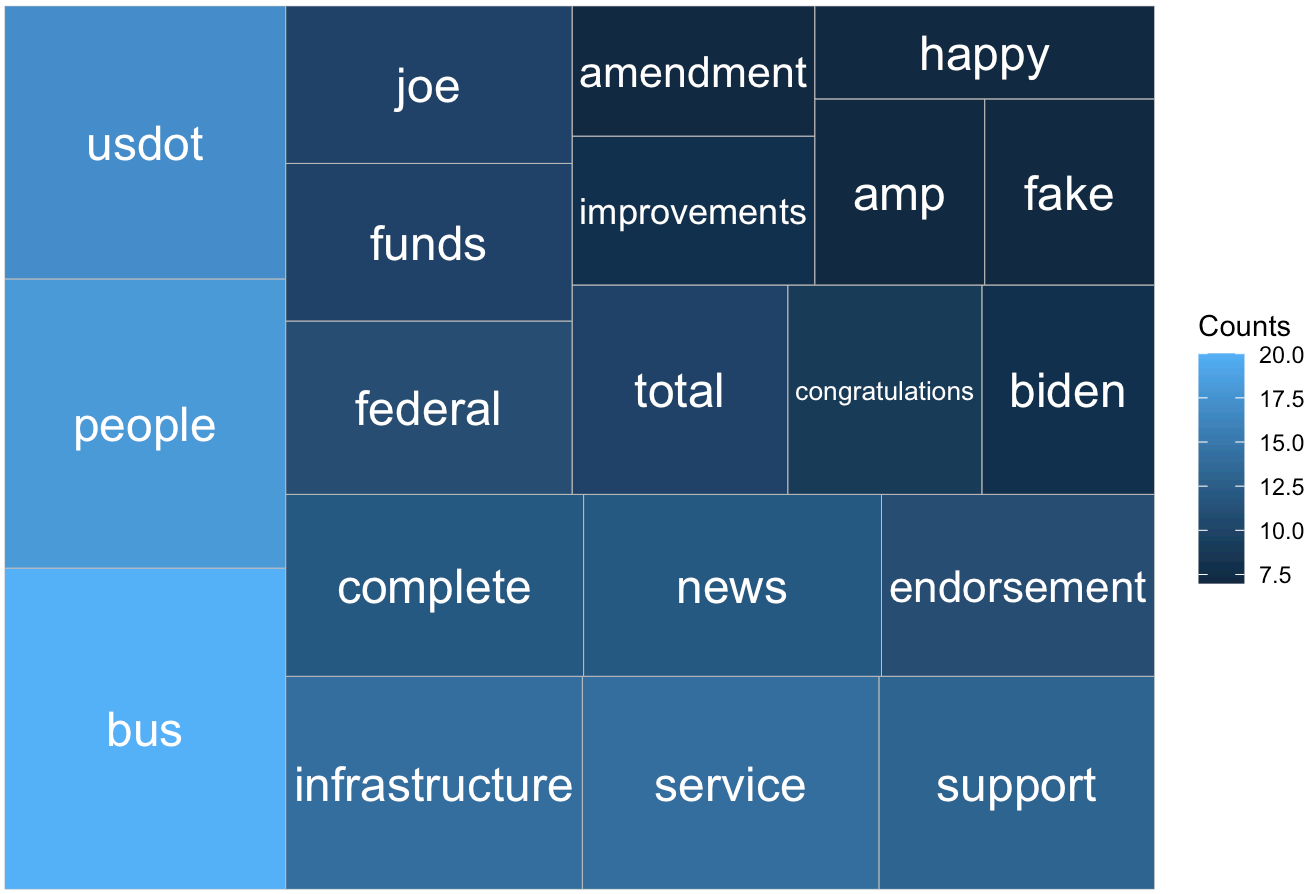
A treemap is another way to visualize frequencies using area.

```
library(treemapify)
```

```
## Warning: package 'treemapify' was built under R version 4.5.2
```

```
ggplot(forCloud %>% slice_head(n = 20),
       aes(area = Counts, fill = Counts, label = EachWord)) +
  geom_treemap() +
  geom_treemap_text(place = "centre", colour = "white") +
  labs(title = "Word frequencies as a treemap")
```

Word frequencies as a treemap

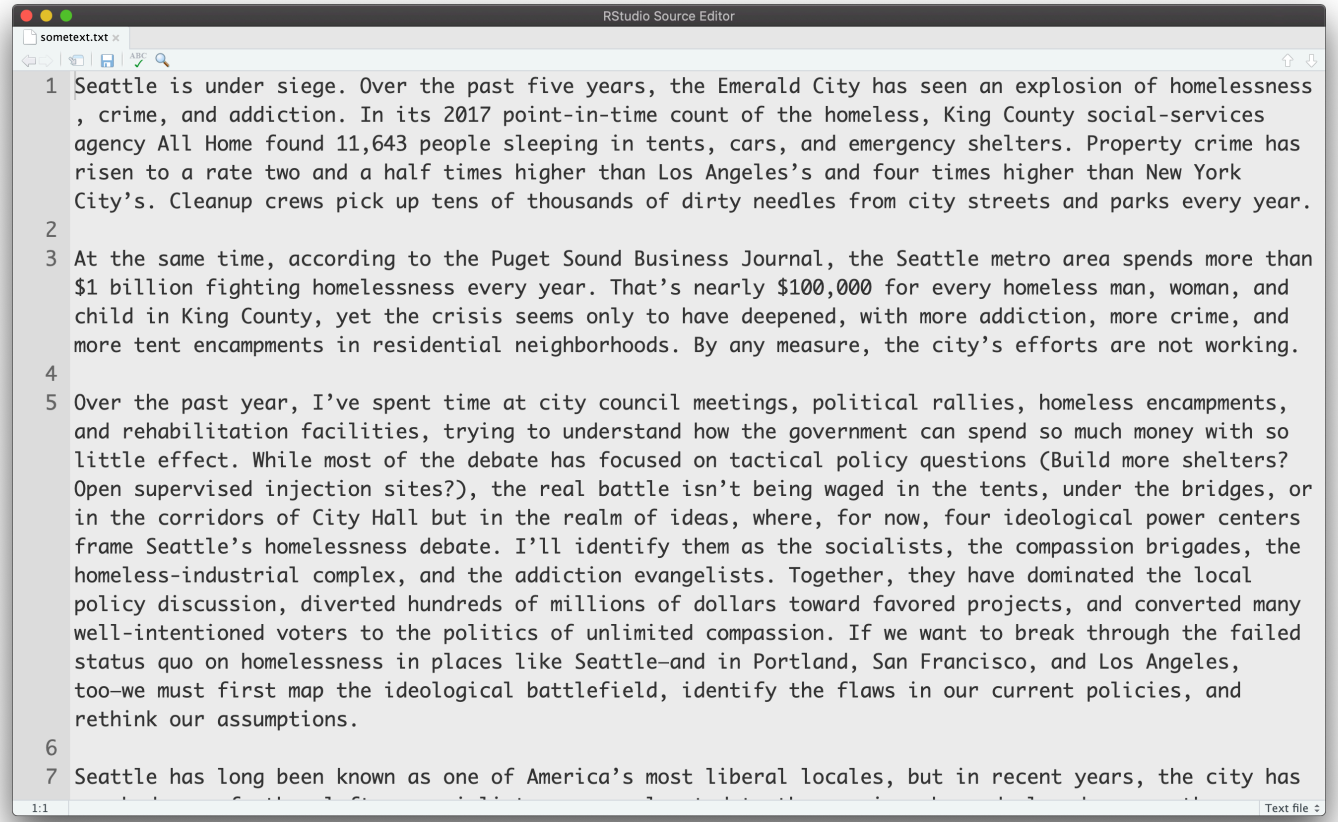


Treemaps are compact and visually appealing, but again, they are not as precise as bars for comparing values.

10. A second example: reading from a TXT file

Not all text data comes from tweets. Sometimes you simply have a text file.

Suppose we have this file:



You can simply save that text as a **txt** file. Let me work with that file:

```
LinkText="https://github.com/DACSS-Visual/basicText/raw/main/sometext.txt"
otherText = read.delim(LinkText, header = FALSE)

## see some
head(otherText)
```

```
##
V1
## 1
Seattle is under siege. Over the past five years, the Emerald City has seen an explosion
of homelessness, crime, and addiction. In its 2017 point-in-time count of the homeless,
King County social-services agency All Home found 11,643 people sleeping in tents, cars,
and emergency shelters. Property crime has risen to a rate two and a half times higher t
han Los Angeles's and four times higher than New York City's. Cleanup crews pick up tens
of thousands of dirty needles from city streets and parks every year.
## 2
At the same time, according to the Puget Sound Business Journal, the Seattle metro area
spends more than $1 billion fighting homelessness every year. That's nearly $100,000 for
every homeless man, woman, and child in King County, yet the crisis seems only to have d
eepened, with more addiction, more crime, and more tent encampments in residential neigh
borhoods. By any measure, the city's efforts are not working.
## 3       Over the past year, I've spent time at city council meetings, political ralli
es, homeless encampments, and rehabilitation facilities, trying to understand how the go
vernment can spend so much money with so little effect. While most of the debate has foc
used on tactical policy questions (Build more shelters? Open supervised injection site
s?), the real battle isn't being waged in the tents, under the bridges, or in the corrid
ors of City Hall but in the realm of ideas, where, for now, four ideological power cente
rs frame Seattle's homelessness debate. I'll identify them as the socialists, the compas
sion brigades, the homeless-industrial complex, and the addiction evangelists. Together,
they have dominated the local policy discussion, diverted hundreds of millions of dollar
s toward favored projects, and converted many well-intentioned voters to the politics of
unlimited compassion. If we want to break through the failed status quo on homelessness
in places like Seattle—and in Portland, San Francisco, and Los Angeles, too—we must firs
t map the ideological battlefield, identify the flaws in our current policies, and rethi
nk our assumptions.
## 4
Seattle has long been known as one of America's most liberal locales, but in recent year
s, the city has marched even further left as socialists, once relegated to the margins,
have declared war on the Democratic establishment. Socialist Alternative city councilwom
an Kshama Sawant claims that the city's homelessness crisis is the inevitable result of
the Amazon boom, greedy landlords, and rapidly increasing rents. As she told Street Root
s News: "The explosion of the homelessness crisis is a symptom of how deeply dysfunction
al capitalism is and also how much worse living standards have gotten with the last seve
ral decades." The capitalists of Amazon, Starbucks, Microsoft, and Boeing, in her Marxia
n optic, generate enormous wealth for themselves, drive up housing prices, and push the
working class toward poverty and despair—and, too often, onto the streets.
## 5 On the surface, this argument has its own internal logic. Advocates point to Zillow
and McKinsey studies that show a high correlation between rent hikes and homelessness in
Seattle, for example. But correlation is not causation, and the survey data paint a rema
rkably different picture. According to King County's point-in-time study, only 6 percent
of homeless people surveyed cited "could not afford rent increase" as the precipitating
cause of their situation, pointing instead to a wide range of other problems—domestic vi
olence, incarceration, mental illness, family conflict, medical conditions, breakups, ev
iction, addiction, and job loss—as bigger factors. Further, while the Zillow study did f
ind correlation between rising rents and homelessness in four major markets—Seattle, Los
Angeles, New York, and Washington, D.C.—it also found that homelessness decreased despit
e rising rents in Houston, Tampa, Chicago, Phoenix, St. Louis, San Diego, Portland, Detr
oit, Baltimore, Atlanta, Charlotte, and Riverside. Rent increases are a real burden for
```

```
the working poor, but the evidence suggests that higher rents alone don’t push people on
to the streets.
## 6
Even in a pricey city like Seattle, most working- and middle-class residents respond to
economic incentives in logical ways: relocating to less expensive neighborhoods, downsiz
ing to smaller apartments, taking in roommates, moving in with family, or leaving the ci
ty altogether. King County is home to more than 1 million residents earning below the me
dian income, and 99 percent of them manage to find a place to live and pay the rent on t
ime. The aggregate-level analyses from Zillow and McKinsey don’t take into account the v
ast number of options available even to the poorest families; for the socialists, “the r
ent is too damn high” explains everything.
```

Notice that a new row is created whenever a new paragraph or line break is found.

Now let us tokenize the text and remove stopwords:

```
otherText_words=otherText |>
               unnest_tokens(output=EachWord,
                             input=V1,# column of the texts
                             token="words") |>
               anti_join(stop_words,
                         by = c("EachWord" = "word"))
# you get
head(otherText_words,20)
```

```
##        EachWord
## 1       seattle
## 2         siege
## 3          past
## 4       emerald
## 5          city
## 6     explosion
## 7  homelessness
## 8         crime
## 9     addiction
## 10         2017
## 11         time
## 12        count
## 13     homeless
## 14         king
## 15       county
## 16       social
## 17     services
## 18       agency
## 19         home
## 20        found
```

Now compute frequencies:

```
txtAsc_descending = otherText_words %>%
  count(EachWord, name = "Counts", sort = TRUE) %>%
  filter(Counts > 4)

# you have
head(txtAsc_descending, 10)
```

```
##        EachWord Counts
## 1      homeless     47
## 2  homelessness     46
## 3          city     40
## 4       seattle     37
## 5     addiction     21
## 6       percent     20
## 7        people     18
## 8        public     18
## 9      services     18
## 10      housing     16
```

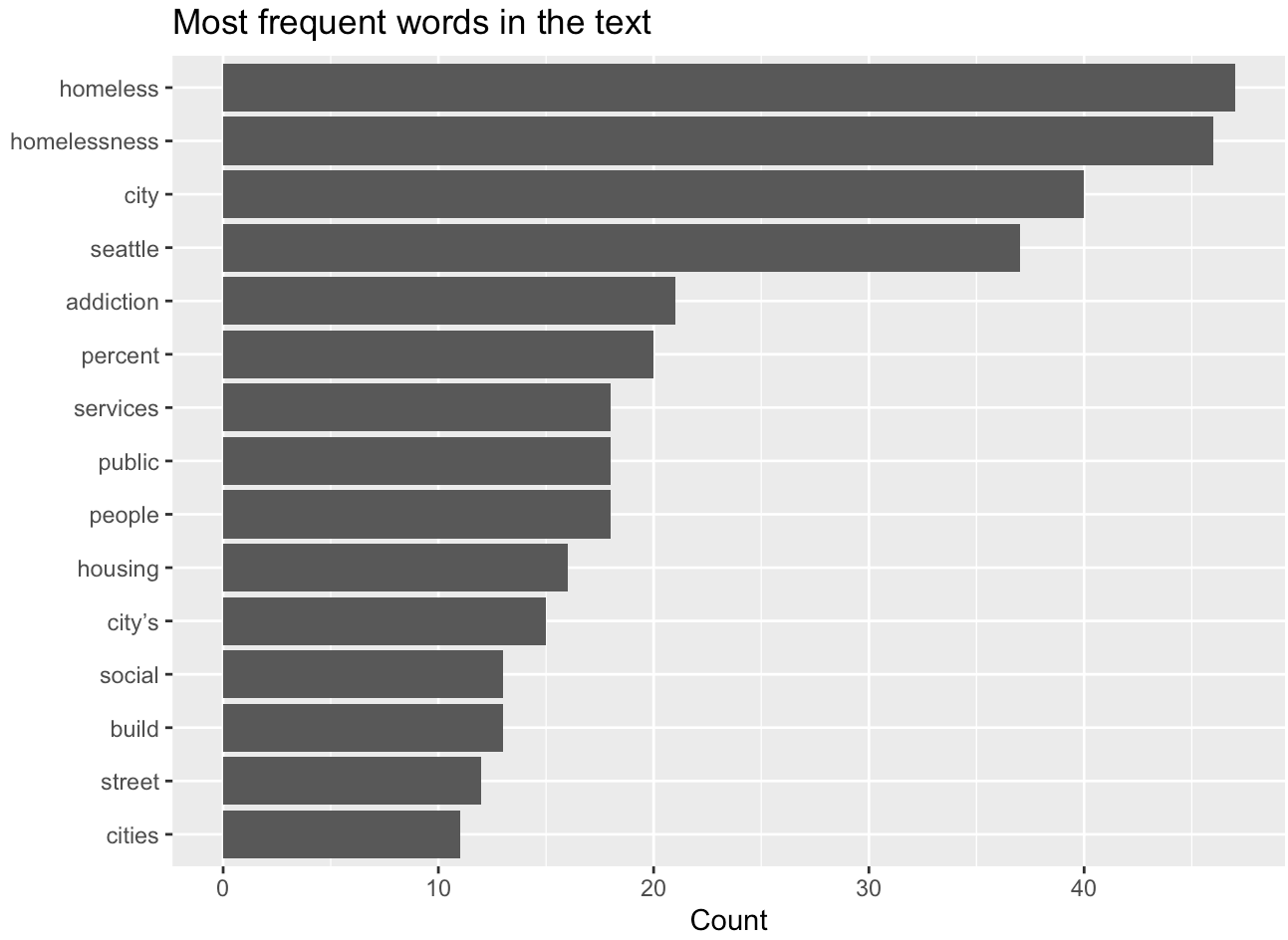
11. Plotting the TXT example

Word cloud

```
ggplot(txtAsc_descending,
       aes(label = EachWord,
           size = Counts,
           color = Counts)) +
  geom_text_wordcloud() +
  scale_size_area(max_size = 10) +
  theme_minimal() +
  scale_color_gradient(low = "red", high = "darkred")
```

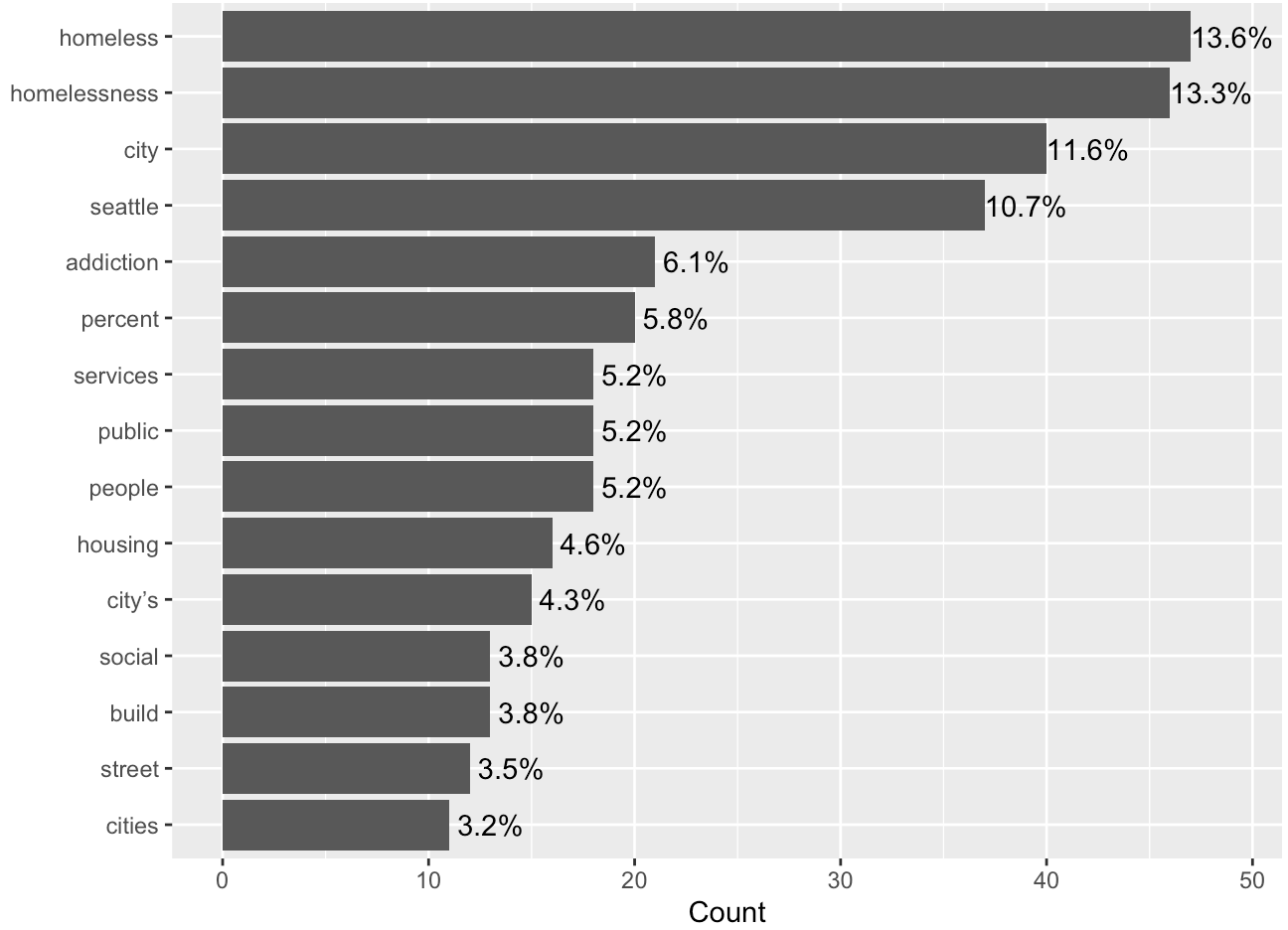
Barplots

```
ggplot(txtAsc_descending %>% slice_head(n = 15),
       aes(x = reorder(EachWord, Counts), y = Counts)) +
  geom_col() +
  coord_flip() +
  labs(x = "", y = "Count",
       title = "Most frequent words in the text")
```



Or, a little better:

```
top15 = txtAsc_descending %>% slice_head(n = 15)

ggplot(top15,
       aes(x = reorder(EachWord, Counts), y = Counts)) +
  geom_col() +
  coord_flip() +
  labs(x = "", y = "Count") +
  geom_text(aes(label = paste0(round(100 * Counts / sum(Counts), 1), "%")),
            nudge_y = 2)
```



This is often more informative than a word cloud because the audience can see both counts and proportions directly.

12. Co-occurrence networks

So far, we have treated words as isolated units. But language also has structure: some words tend to appear together.

A simple way to explore this is through bigrams, that is, pairs of words that appear next to each other.

```
library(tidyr)
library(igraph)
```

```
##
## Attaching package: 'igraph'
```

```
## The following object is masked from 'package:tidyr':
##
##     crossing
```

```
## The following objects are masked from 'package:dplyr':
##
##     as_data_frame, groups, union
```

```
## The following objects are masked from 'package:stats':
##
##     decompose, spectrum
```

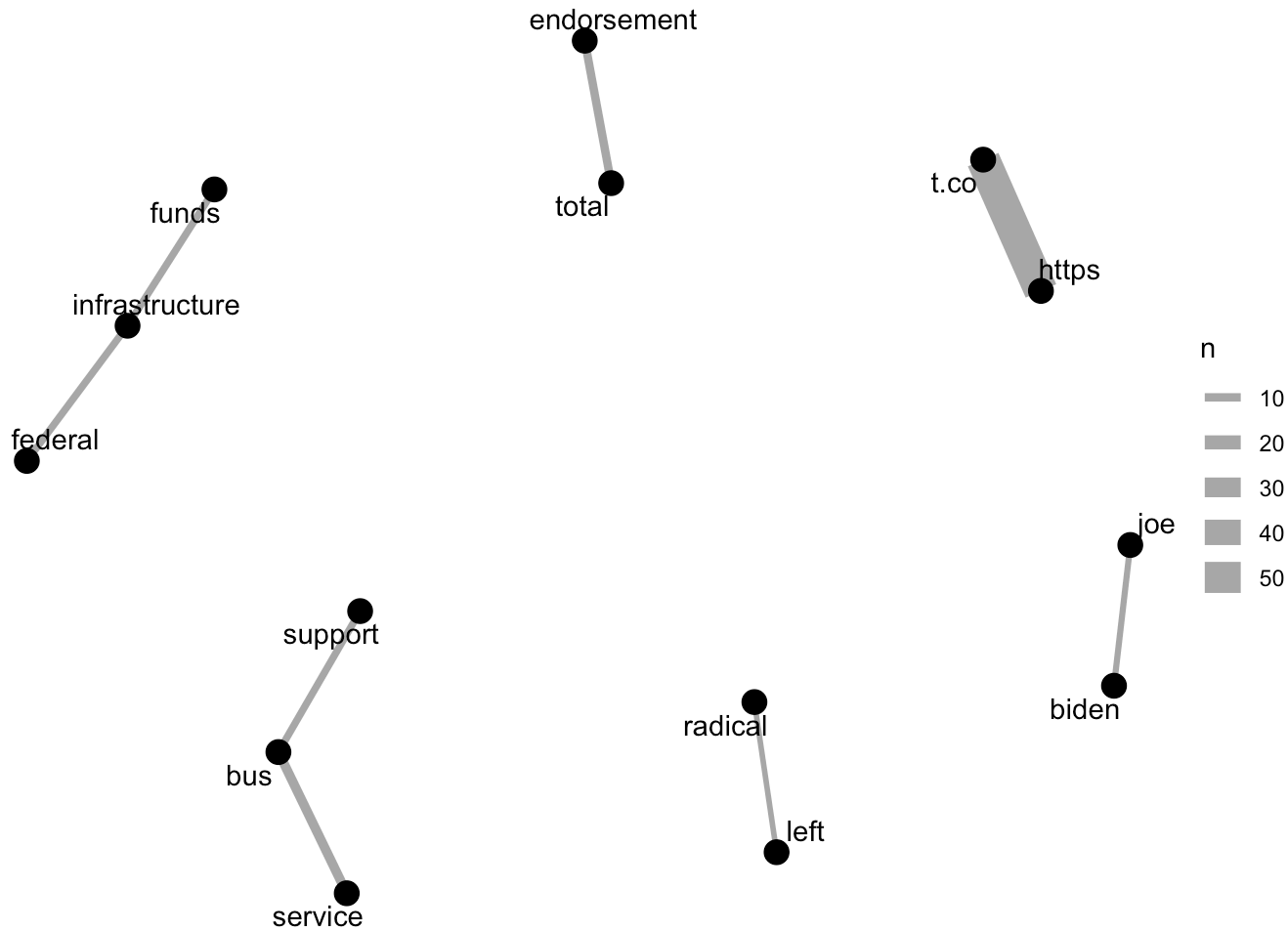
```
## The following object is masked from 'package:base':
##
##     union
```

```
library(ggraph)

bigrams = SomeTrumpTweets %>%
  unnest_tokens(bigram, text, token = "ngrams", n = 2)

bigram_counts = bigrams %>%
  separate(bigram, into = c("word1", "word2"), sep = " ") %>%
  filter(!word1 %in% stop_words$word,
         !word2 %in% stop_words$word) %>%
  count(word1, word2, sort = TRUE) %>%
  filter(n >= 5)

head(bigram_counts, 10)
```

```
##            word1          word2  n
## 1          https           t.co 54
## 2            bus        service 12
## 3          total    endorsement 10
## 4        federal infrastructure  8
## 5 infrastructure          funds  8
## 6        support            bus  8
## 7            joe          biden  6
## 8        radical           left  5
```

```
bigram_graph = graph_from_data_frame(bigram_counts)

ggraph(bigram_graph, layout = "fr") +
  geom_edge_link(aes(width = n), alpha = 0.4) +
  geom_node_point(size = 4) +
  geom_node_text(aes(label = name), repel = TRUE) +
  theme_void()
```

```
## Warning: The `trans` argument of `continuous_scale()` is deprecated as of ggplot2 3.
5.0.
## ℹ Please use the `transform` argument instead.
## This warning is displayed once per session.
## Call `lifecycle::last_lifecycle_warnings()` to see where this warning was
## generated.
```



This plot does not focus on word frequency alone. Instead, it helps reveal relationships between words.

That is often more meaningful than treating words separately.

13. Topic models

Sometimes we want to move from individual words to broader themes in a collection of documents. Topic models help us do that.

The basic idea is simple: instead of asking “Which words are frequent?”, topic models ask:

Which words tend to appear together across documents, suggesting a latent topic?

Here, each tweet is treated as a document.

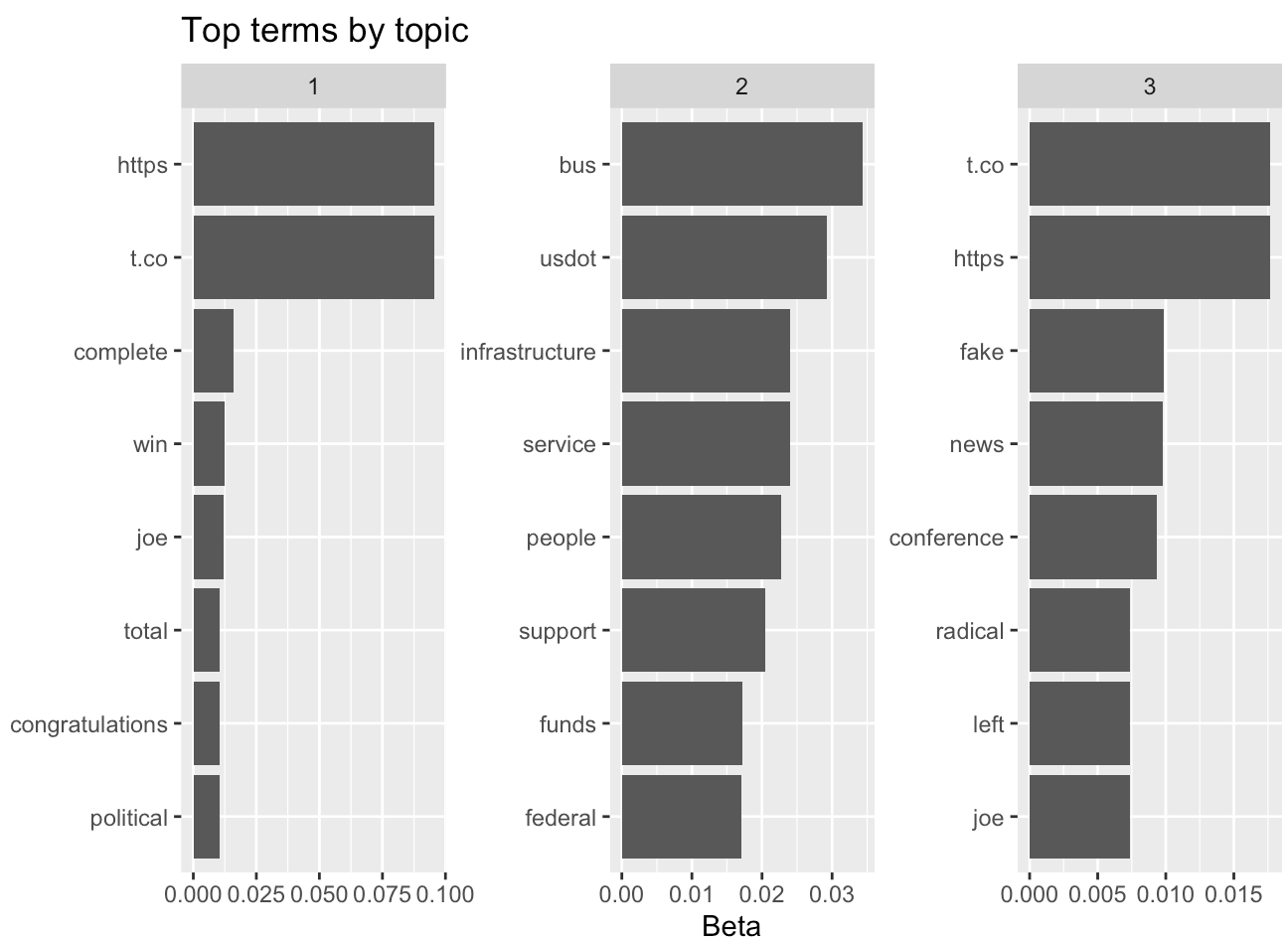
```
library(topicmodels)

tweet_words = SomeTrumpTweets %>%
  mutate(doc_id = row_number()) %>%
  unnest_tokens(word, text) %>%
  anti_join(stop_words, by = "word")

tweet_dtm = tweet_words %>%
  count(doc_id, word) %>%
  cast_dtm(document = doc_id, term = word, value = n)

lda_model = LDA(tweet_dtm, k = 3, control = list(seed = 1234))
```

Let us extract the most important terms in each topic:

```
library(tidytext)

topics_top_terms = tidy(lda_model, matrix = "beta") %>%
  group_by(topic) %>%
  slice_max(beta, n = 8) %>%
  ungroup()
```

To plot them nicely, we use a helper from tidytext:

```
library(tidytext)

ggplot(topics_top_terms,
       aes(x = reorder_within(term, beta, topic),
           y = beta)) +
  geom_col() +
  facet_wrap(~topic, scales = "free") +
  scale_x_reordered() +
  coord_flip() +
  labs(x = "", y = "Beta",
       title = "Top terms by topic")
```



This is not just a visualization technique. It is also a modeling strategy for discovering themes in text.

14. Interpreting text plots carefully

Text graphics can be attractive, but interpretation requires caution.

A frequent word is not necessarily an important idea. It may be:

- a repeated name
- a slogan
- a response to current events
- part of criticism or support
- a word used sarcastically

So, frequency does not automatically imply meaning.

Also, different forms of the same idea may appear as different words. For example:

- run
- runs
- running

This is why more advanced text analysis often uses **stemming** or **lemmatization**.

Single-word plots are a useful starting point, but they do not capture all the complexity of language.

15. What each plot is good for

Here is a useful summary:

- Ranked bar chart: best for precise comparison
- Lollipop chart: similar to bars, visually lighter
- Word cloud: engaging, but weak for exact comparison
- Treemap: compact view of relative frequency
- Co-occurrence network: reveals relationships between words
- Topic model plot: reveals broader themes across documents

So the choice of plot depends on the goal.

- If your goal is comparison, bars are usually best.
- If your goal is engagement, a word cloud may help.
- If your goal is structure or themes, networks and topic models are more useful.

16. Final remark

Text visualization is powerful, but language is complex.

Good text graphics require:

- good preprocessing
- good visualization choices
- careful interpretation

In other words, text visualization is not only about making words look attractive. It is about transforming language into data without forgetting that language always carries context, ambiguity, and meaning.